Running docker after setting up the docker-compose.yml and checking the containers currently running

```
PS C:\Users\ASUS TUF\Desktop\FUSE\sdlc> docker compose up -d
time="2026-05-09T21:10:46+05:45" level=warning msg="project has been loaded without an explicit name from a symlink. Using name \"sdlc\""
[+] up 2/2
 ✓ Network sdlc_default    Created                                                              0.0s
 ✓ Container fuse_postgres Started                                                              0.4s
PS C:\Users\ASUS TUF\Desktop\FUSE\sdlc> docker ps -a
CONTAINER ID   IMAGE         COMMAND                  CREATED          STATUS          PORTS                                         NAMES
119668526eb8   postgres:16   "docker-entrypoint.s…"   24 seconds ago   Up 23 seconds   0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp   fuse_postgres
PS C:\Users\ASUS TUF\Desktop\FUSE\sdlc>
```

Entering the container

```
PS C:\Users\ASUS TUF\Desktop\FUSE\sdlc> docker exec -it fuse_postgres /bin/bash
root@119668526eb8:/# whoami
root
root@119668526eb8:/#
```

Connected to PostgreSQL inside the container

```
root@119668526eb8:/# psql -U fuseadmin -d fusedb
psql (16.13 (Debian 16.13-1.pgdg13+1))
Type "help" for help.

fusedb=#
```

Verify Tables in the database

```
fusedb=# \dt
             List of relations
 Schema |     Name      | Type  |   Owner
--------+---------------+-------+-----------
 public | customers     | table | fuseadmin
 public | employees     | table | fuseadmin
 public | offices       | table | fuseadmin
 public | orderdetails  | table | fuseadmin
 public | orders        | table | fuseadmin
 public | payments      | table | fuseadmin
 public | productlines  | table | fuseadmin
 public | products      | table | fuseadmin
(8 rows)
```

Run Queries

```
fusedb=# SELECT COUNT(*) FROM customers;
 count
-------
   122
(1 row)
```

```
fusedb=# \x
Expanded display is on.
fusedb=# SELECT * FROM customers LIMIT 2;
-[ RECORD 1 ]----------+-------------------
customerNumber         | 103
customerName           | Atelier graphique
contactLastName        | Schmitt
contactFirstName       | Carine
phone                  | 40.32.2555
addressLine1           | 54, rue Royale
addressLine2           |
city                   | Nantes
state                  |
postalCode             | 44000
country                | France
salesRepEmployeeNumber | 1370
creditLimit            | 21000.00
-[ RECORD 2 ]----------+-------------------
customerNumber         | 112
customerName           | Signal Gift Stores
contactLastName        | King
contactFirstName       | Jean
phone                  | 7025551838
addressLine1           | 8489 Strong St.
addressLine2           |
city                   | Las Vegas
state                  | NV
postalCode             | 83030
country                | USA
salesRepEmployeeNumber | 1166
creditLimit            | 71800.00
```

Short Reflection

Config (Factor III)

Question: Why is .env better than writing passwords in code?

Using a .env file is better than writing passwords directly in the code because it keeps sensitive information such as database usernames and passwords separate from the application logic. This improves security since credentials are not exposed inside the source code. It also makes the application easier to manage because configuration values can be changed without modifying the code itself. Additionally, it reduces the risk of accidentally pushing sensitive data to public repositories like GitHub.

Backing Services (Factor IV)

Question: Why is treating the database as a separate service useful?

Treating the database as a separate service is useful because it decouples the database from the main application. This means the database can be updated, replaced, or scaled independently without affecting the application code. It also improves flexibility and maintainability, as the application does not depend on a tightly integrated database system. This separation follows modern architecture practices and makes deployment more reliable.

Dev/Prod Parity (Factor X)

Question: Why does Docker make development and production similar?

Docker makes development and production environments similar by packaging the application and its dependencies into containers. This ensures that the same PostgreSQL setup used in development can also run in production without changes. As a result, it reduces environment-related issues such as configuration mismatches and “works on my machine” problems. This consistency improves reliability and makes deployment smoother.